HACKTHEBOX

LINUX PROCESS INJECTIONS & DETECTIONS

CHEAT SHEET

ELF Executable Header

Field	Short description
`e_ident`	16-byte ID (magic `0x7F 'E' 'L' 'F'`, class `ELFCLASS64`, endianness, version, OSABI, ABI version)
`e_type`	File type (`ET_EXEC`, `ET_DYN`, `ET_REL`, `ET_CORE`, etc.)
`e_machine`	Target architecture (`EM_X86_64`, `EM_AARCH64`, …)
`e_version`	File version (usually `EV_CURRENT` = 1)
`e_entry`	Entry point virtual address
`e_phoff`	Offset to program header table
`e_shoff`	Offset to section header table
`e_flags`	Processor-specific flags
`e_ehsize`	ELF header size (bytes)
`e_phentsize`	Size of one program header entry
`e_phnum`	Number of program header entries
`e_shentsize`	Size of one section header entry

Field	Short description
`e_shnum`	Number of section header entries
`e_shstrndx`	Section index of the section-name string table

ELF Sections

Section Name	Purpose
`.interp`	Points to the program that loads and runs the application.
`.dynsym`	Lists symbols needed for shared libraries and runtime linking.
`.dynstr`	Stores the names of symbols used in dynamic linking.
`.rela.dyn`	Holds data for updating addresses during dynamic linking.
`.rela.plt`	Contains address updates for the Procedure Linkage Table (PLT).
`.init`	Code that runs before the main program starts.
`.plt`	Procedure Linkage Table for handling function calls dynamically.
`.plt.got`	Global Offset Table entries for PLT (used in some systems).
`.plt.sec`	PLT entries compatible with Intel Control-flow Enforcement Technology (CET). To be seen as an extension of the standard PLT.
`.text`	The main program code that gets executed.
`.fini`	Code that runs after the main program finishes.
`.rodata`	Stores read-only data, like constants and text strings.
`.eh_frame_hdr`	Header for data used in exception handling and stack unwinding.
`.eh_frame`	Data for managing exceptions and unwinding the stack.

HTB ACADEMY CHEATSHEET

Section Name	Purpose
`.init_array`	List of pointers to functions that run at program startup.
`.fini_array`	List of pointers to functions that run when the program ends.
`.dynamic`	Information for the runtime linker to manage dynamic linking.
`.got`	Global Offset Table for resolving symbol addresses dynamically.
`.got.plt`	GOT entries specifically for PLT address updates.
`.data`	Stores initialized data that the program can modify.
`.bss`	Holds uninitialized data, set to zero when the program starts.
`.shstrtab`	Stores names of sections in the section header table.

GDB & Utility Commands

Command	Notes / Purpose
`gcc program.c -o program`	Build binary (default linker behavior; may be eager). Use `-Wl,-z,lazy` to force lazy linking.
`gcc program.c -o program -Wl,-z,lazy`	Build with lazy symbol resolution.
`readelf -d program | grep "FLAGS)"`	Show dynamic flags (e.g. `BIND_NOW`).
`LD_DEBUG=bindings ./program`	Trace dynamic linker binding activity at runtime.
`objdump -s -j .plt -d ./program -M intel`	Static dump/disassembly of `.plt` (use other section names like `.plt.sec`, `.got`, `.plt.got` as needed).
`gdb -q ./program`	Start gdb quietly.
`(gdb) set disassembly-flavor intel`	Use Intel syntax for disassembly (x86_64).

Command	Notes / Purpose
`(gdb) break puts@plt`	Break at a symbol-specific location (substitute any symbol or address).
`(gdb) run` or `(gdb) r`	Start program under the debugger.
`(gdb) disas $pc`	Disassemble around the current PC/RIP.
`(gdb) si`	Single-step one machine instruction (follow instruction-level flow).
`(gdb) x/20i $pc`	Dump instructions (adjust count/address to inspect code regions).
`(gdb) x/3i <addr>`	Disassemble a few instructions at `<addr>` (useful for headers/stubs).
`(gdb) x/gx <addr>`	Read an 8-byte quantity at `<addr>` (GOT entries, pointers, addresses).
`(gdb) info sym <addr>`	Resolve address → symbol (shows if an address maps to a known symbol).
`(gdb) break _dl_runtime_resolve_xsavec`	Break inside a runtime resolver (example for dynamic linking; resolver name may vary).
`(gdb) continue` or `(gdb) c`	Continue execution until next breakpoint.
`(gdb) finish`	Run until current function returns (useful after hitting a resolver/trampoline).
`(gdb) disas <addr>`	Disassemble function/code at `<addr>` (e.g. resolved library function).
`(gdb) info proc mappings`	Show process memory mappings to correlate files → in-memory addresses.
`(gdb) info files`	Show files and load addresses known to gdb.

HTB ACADEMY CHEATSHEET

`/proc/[pid]/` Directory Overview

Entry	Details
`cmdline`	Full command line (argv, `\0`-separated).
`cwd`	Symlink to the current working directory.
`environ`	Process environment variables.
`exe`	Symlink to the executed binary.
`fd/`	Open file descriptors → symlinks to files/sockets/pipes.
`fdinfo/`	Per-FD (File Descriptor) info: pos, flags.
`maps`	Virtual memory map (code, heap, stack, libs).
`mem`	Raw process memory (read/write via ptrace/root).
`mounts`	Filesystems mounted (process view).
`mountinfo`	Extended mount info (IDs, opts, namespaces).
`net/`	Network state (sockets, TCP, UDP, etc.).
`ns/`	Namespace handles (mnt, pid, uts, net, user…).
`numa_maps`	NUMA placement of memory pages.
`oom_score`	Value used by OOM killer (higher → more likely to be killed).
`root`	Symlink to process root (`chroot` aware).
`smaps`	Detailed memory stats per VMA (RSS, PSS, swap).
`stat`	Raw stats (PID, PPID, state, utime, stime, faults…).
`statm`	Memory stats (size, resident, shared, text, data).
`status`	Human-readable summary (UIDs, GIDs, threads, mem).

Entry	Details
`task/`	Subdirectories per thread (each like `/proc/[pid]/`).
`wchan`	Kernel function where the task sleeps.

x86-64 Linux Calling Convention (Argument Registers)

Argument	Placement
1st	`RDI`
2nd	`RSI`
3rd	`RDX`
4th	`RCX`
5th	`R8`
6th	`R9`
7th+	`Stack`

`Ptrace` Options

`ptrace` Option	Description
`PTRACE_TRACEME`	Indicate child will be traced by parent (before exec).
`PTRACE_ATTACH`	Attach to an existing process for tracing.
`PTRACE_DETACH`	Detach from tracee and resume normal execution.
`PTRACE_CONT`	Continue tracee execution, optionally delivering a signal.
`PTRACE_SINGLESTEP`	Execute a single instruction and stop.

`ptrace` Option	Description
`PTRACE_SYSCALL`	Stop tracee at every syscall entry and exit.
`PTRACE_GETREGS`	Read general-purpose registers.
`PTRACE_SETREGS`	Modify general-purpose registers.
`PTRACE_POKEUSER`	Write a word into tracee's user area (fragile; prefer GET/SETREGS).
`PTRACE_GETFPREGS`	Read floating-point / SSE registers.
`PTRACE_SETFPREGS`	Modify floating-point / SSE registers.
`PTRACE_GETREGSET`	Access architecture-specific register groups.
`PTRACE_SETREGSET`	Modify register groups.
`PTRACE_GETEVENTMSG`	Get message for certain ptrace events (fork, exec, etc.).
`PTRACE_GETSIGINFO`	Retrieve detailed info about the stopping signal.
`PTRACE_SETSIGINFO`	Modify signal to deliver to tracee.
`PTRACE_INTERRUPT`	Interrupt tracee immediately without signal.
`PTRACE_LISTEN`	Put tracee in event-listening mode.
`PTRACE_SEIZE`	Attach without stopping, with fine-grained options.
`PTRACE_SETOPTIONS`	Configure tracing behavior (syscall stops, fork/exec/exit events).
`PTRACE_GET_SYSCALL_INFO`	Get current syscall info (number, args, return value).

HTB ACADEMY CHEATSHEET

`Auditctl` Common Parameters

Parameter	Description & Usage	Example & Notes
`-a`	Append a rule to a list. Format: `-a <action>,<list>`.	`-a always,exit`. Always log events; common lists: exit, task.
`-A`	Prepend a rule to a list (higher priority).	`-A always,exit`. Ensures rule precedence.
`-S`	Specify syscalls by name or number. Multiple allowed.	`-S execve -S openat`. Use `ausyscall --dump` to find names.
`-F`	Add filters to narrow scope: `<field>=<value>` or `!=`.	`-F arch=b64 -F uid!=0`. Multiple filters use AND logic.
`-p`	Define file access permissions: r, w, x, a.	`-w /etc/passwd -p wa`. Paired with `-w`.
`-w`	Watch files or directories for access/modification.	`-w /etc/ld.so.preload -p wa`. Paths must be absolute.
`-k`	Assign a key label to rules for filtering.	`-k ld_preload_hijack`. Max 31 characters, no spaces.
`-D`	Delete all existing rules.	`-D`. Typically at the start of a rules file.
`-b`	Set backlog buffer size (queued audit records).	`-b 8192`. Larger buffers reduce dropped events.
`-f`	Set failure mode: 0=silent, 1=printk, 2=panic.	`-f 1`. Use 2 to halt on failure.
`-e`	Enable, disable, or lock auditing: 0,1,2.	`-e 2`. Locks config until reboot.
`-i`	Ignore case in string comparisons.	`-i -F path=/ETC/PASSWD`. Useful for case-insensitive paths.
`-c`	Continue processing rules after a match.	`-c`. Default is stop after first match.

HTB ACADEMY CHEATSHEET

HTB ACADEMY CHEATSHEET

HTB ACADEMY CHEATSHEET